

THE PART 2 PLAN

Phase 0: Read the ISA and Produce Internal Documentation

Before writing code, extract from the spec:

Registers

```
pc
sp
tm
ga
gb
gc
gd
ge
gf
gg
gh
```

11 registers total.

Create a Python mapping:

```
REG_PC = 0
REG_SP = 1
...
REG_GH = 10
```

or

```
regs = {
    "pc": 0,
    ...
}
```

MMIO Map

0x0000 - 0x8fff	RAM
0x9000 - 0x9800	Display
0x9801	Button
0x9802	Tick Counter
0x9803	RNG
0xa000 - 0xffff	Stack RAM

Create constants immediately.

Instruction Table

Create one master table:

```
INSTRUCTIONS = {  
    0x00: "jmp",  
    0x01: "set",  
    ...  
}
```

This will later drive:

- decoding
 - disassembly display
 - execution
-

Phase 1: Binary Loader

Goal:

```
Read .252bin  
Load memory  
Exit
```

No emulator yet.

Memory:

```
memory = [0] * 65536
```

because address space is 2^{16} words.

Read binary:

```
with open(path, "rb") as f:
```

Load little-endian words.

Verify:

```
memory[0]  
memory[1]  
...
```

with test files.

Phase 2: Register File and CPU Skeleton

Implement:

```
class CPU:
```

with

```
regs  
memory  
tick_counter  
rng
```

Initialize:

```
pc = 0  
sp = 0xffff
```

(or whatever the spec/testing assumes)

Create:

```
step()
```

which will eventually execute one instruction.

At this point:

```
cpu.step()
```

does nothing.

Phase 3: Instruction Fetch

Goal:

Fetch opcode correctly.

Read first word:

```
instr_word = memory[pc]
```

Extract:

```
opcode = instr_word & 0x1f
op2     = (instr_word >> 5) & 0x1f
op1     = (instr_word >> 10) & 0x3f
```

Create:

```
decode()
```

returning

```
DecodedInstruction(...)
```

Phase 4: Instruction Width Detection

Critical phase.

Arch-252 has:

```
Type 1 = 2 bytes
Type 2 = 4 bytes
Type 3 = 6 bytes
Type 4 = 2 bytes
```

You need:

```
instruction_width()
```

because:

```
NextPC
NextNextPC
```

depend on it.

Many branches will break if this is wrong.

Phase 5: Operand Resolution

This is the hardest architectural component.

Create:

```
read_operand(opcode)
write_operand(opcode, value)
```

Support:

```
register
@register
@register+k
push
pop
peek
```

```
@peek+k  
literal  
@literal
```

Everything else should call these helpers.

Goal:

Instruction handlers should look like:

```
a = read(op1)  
b = read(op2)  
  
write(op1, a + b)
```

not:

```
if op1 == ...  
if op2 == ...
```

everywhere.

Phase 6: Arithmetic Instructions

Implement first:

```
set  
add  
sub  
and  
or  
xor
```

These are easiest.

Verify:

```
register-register  
register-memory  
memory-register
```

cases.

Phase 7: Remaining ALU Instructions

Add:

```
mulu  
muls  
divu  
divs  
modu  
mods  
sll  
srl  
sra
```

Pay special attention to:

```
tm
```

overflow behavior.

This phase will likely consume most debugging time.

Phase 8: Branches and Control Flow

Implement:

```
jmp  
ifeq  
ifne  
ifgtu  
ifgts  
ifltu  
iflts  
ifany  
ifnon
```

Build a helper:

```
next_pc()  
next_next_pc()
```

because every branch uses them.

Phase 9: seti / setd

Implement separately.

These have special behavior:

```
gg++  
gh++
```

or

```
gg--  
gh--
```

Keep isolated from ordinary instructions.

Phase 10: MMIO Layer

Create:

```
memory_read(addr)  
memory_write(addr)
```

Never access memory directly again.

Inside:

```
if addr == BUTTON:
```

etc.

Implement:

Button

```
0 or 1
```

Tick Counter

Returns:

```
tick & 0xffff
```

RNG

Use:

```
random.Random(seed)
```

Phase 11: Headless Emulator

Before touching Pyxel:

Create:

```
python emulator.py file.bin 1 123
```

and run entirely in terminal.

No graphics.

No GUI.

Just:

```
for i in range(1000):  
    cpu.step()
```

This phase should get all instruction semantics correct.

Phase 12: Display Buffers

Implement:

```
front_buffer  
back_buffer
```

1024 pixels each.

Implement:

```
swap_buffers()
```

exactly as specified.

Test separately.

No Pyxel yet.

Phase 13: Pyxel Window

Now finally:

```
pyxel.init(...)
```

Render:

```
32x32 grid
```

Initially:

```
draw random colors
```

to verify rendering.

Phase 14: Color Quantization

Implement nearest-color search over the DawnBringer palette.

Create:

```
quantize(rgb)
```

Test separately.

Phase 15: Connect MMIO Display to Pyxel

Memory writes:

```
0x9000-0x9800
```

must update inactive buffer.

Render active buffer.

At this point:

Flappy Bird can eventually display graphics.

Phase 16: Information Panel

Implement:

```
ipf
seed
instructions executed
tick counter
pc
current instruction
all registers
```

This is mostly UI work.

Leave room for the recording feature.

Phase 17: Video Feature (Do Last)

Add:

```
sp-3
sp-2
sp-1
sp
sp+1
sp+2
sp+3
```

display.

Do not implement this until recording day.

It's almost completely independent of the rest of the emulator.

Recommended Timeline

Week 1

- ISA tables
- memory
- loader
- fetch/decode

Week 2

- operand system
- arithmetic instructions
- branches

Week 3

- MMIO
- display buffers
- headless testing

Week 4

- Pyxel frontend
- information panel

- polish

Final Week

- recording feature
- bug fixes
- integration with Part 3

This order minimizes risk because the CPU semantics are verified before any GUI code gets involved. Most emulator projects become difficult when execution bugs and rendering bugs are introduced simultaneously.